

HomiCSx: An end-to-end FEniCSx framework for finite-element computational homogenization

Amir Reza Tahouni*

Independent researcher, Tehran, Iran

Abstract

HomiCSx is an open-source Python framework for first-order finite-element computational homogenization of heterogeneous materials. It treats geometry generation, periodic-conforming meshing, material assignment, finite-element formulation, linear or finite-strain homogenization, workflow customization, and essential result extraction as one reproducible pipeline. The supported core includes two- and three-dimensional particulate cells, triangular or tetrahedral Gmsh meshes, multiphase linear elasticity, compressible Neo-Hookean materials, and a finite-strain generalized-Maxwell model. Periodic fluctuation constraints are imposed through DOLFINx multipoint constraints; user-defined load cases, nonlinear materials, and hooks allow research-specific extensions without rewriting the driver. Verification combines analytical homogeneous patch tests, constitutive energy-gradient checks, mesh-refinement gates, and deterministic regression tests. Independent Abaqus comparisons use conventional macroscopic quantities and cover linear heterogeneous cells, finite-strain hyperelastic shear, and homogeneous and heterogeneous viscoelastic relaxation. Maximum discrepancies in the canonical linear suite are below 0.55%; heterogeneous viscoelastic stress-history discrepancies are below 1.36% by the peak-normalized maximum measure for the tested cells. HomiCSx is intended to make customized computational-homogenization studies easier to construct, inspect, reproduce, and extend while stating a deliberately bounded support and maintenance scope.

Keywords: computational homogenization, finite element method,

*Corresponding author

Email address: tahouniamirreza@gmail.com (Amir Reza Tahouni)

1. Motivation and significance

First-order computational homogenization obtains macroscopic constitutive response from microscopic boundary-value problems under scale separation [4, 5]. Its finite-element realization requires a chain of choices that are often implemented in separate research scripts: constructing a representative cell, preserving periodic geometry, producing compatible opposite-boundary meshes, assigning phases, enforcing periodic constraints, solving multiple macroscopic load cases, and reducing microscopic fields to macroscopic quantities. This fragmentation makes it difficult to adapt a workflow without also weakening its traceability.

HomiCSx provides these steps through a common set of Python data objects and drivers built on DOLFINx and UFL [1, 2], with geometry and meshing provided by Gmsh [3]. Its main contribution is not a new homogenization theory. It is an extensible end-to-end implementation in which the geometry, discretization, constitutive response, macroscopic loading, nonlinear solution, and result extraction remain explicit and replaceable. This is useful for studies that require systematic microstructure generation or research-specific operations during a nonlinear homogenization history.

The software supports linear and finite-strain problems in two and three dimensions. Research-specific behavior can be introduced through custom material implementations, callable load cases, and typed hooks invoked at documented points in the nonlinear driver. The package therefore occupies a middle ground between one-off finite-element scripts and closed workflows that offer limited intervention in the solution sequence.

2. Software description

2.1. Architecture and workflow

The workflow has five cooperating layers. The geometry layer creates seeded, periodic particulate cells or accepts explicitly constructed inclusions. The mesh layer builds the corresponding OpenCASCADE model, assigns physical tags, and generates triangular or tetrahedral meshes. Opposite boundaries are meshed with translated Gmsh periodic constraints so their

nodes match. The material layer maps phase identifiers to constitutive objects. The finite-element layer constructs linear or nonlinear periodic fluctuation problems and applies multipoint constraints. Finally, homogenization drivers execute standard or user-defined macroscopic load cases and return macroscopic histories and optional fields.

The core geometry types include circular and elliptical inclusions in 2D and spherical and spheroidal inclusions in 3D, with monodisperse or polydisperse sampling and clearance control. Periodic images retain source identity so that boundary-split particles are represented consistently. Geometry generation is seedable for reproducibility. The publication-supported discretization path uses triangles or tetrahedra; forced quad/hex workflows are outside the current support boundary.

2.2. Homogenization formulation

For linear elasticity, the microscopic displacement is decomposed into a macroscopic affine field and a periodic fluctuation. Elementary macroscopic strain probes recover the effective stiffness from volume-averaged stress. In finite strain, the deformation gradient is decomposed as

$$\mathbf{F}(\mathbf{X}) = \bar{\mathbf{F}} + \nabla \tilde{\mathbf{u}}(\mathbf{X}), \quad (1)$$

where the fluctuation is periodic on opposite cell boundaries. The nonlinear driver solves incremental equilibrium with adaptive stepping and records volume-averaged first Piola–Kirchhoff stress, energy, deformation Jacobian, and optional tangent information.

Built-in nonlinear behavior includes compressible Neo-Hookean elasticity and a generalized-Maxwell solid. Each Maxwell branch stores a cellwise viscous metric and advances it with an exponential recurrence based on the current right Cauchy–Green tensor and branch relaxation time. The resulting algorithmic nonequilibrium stress participates directly in the weak residual, and its Newton Jacobian is obtained by automatic differentiation. Previous converged viscous metrics remain fixed during a global solve and are committed only after convergence, preserving step-retry semantics.

2.3. Customization and outputs

Users can define macroscopic deformation histories as Python callables and can derive new nonlinear material classes through the public material interface. Hooks provide controlled access before and after documented stages

of the homogenization loop. They have explicit ordering and state scopes and can be used to collect fields, compute application-specific metrics, or implement additional workflow logic. Core outputs include effective stiffness, macro stress/strain/energy histories, Jacobian histories, and XDMF field exports for external post-processing.

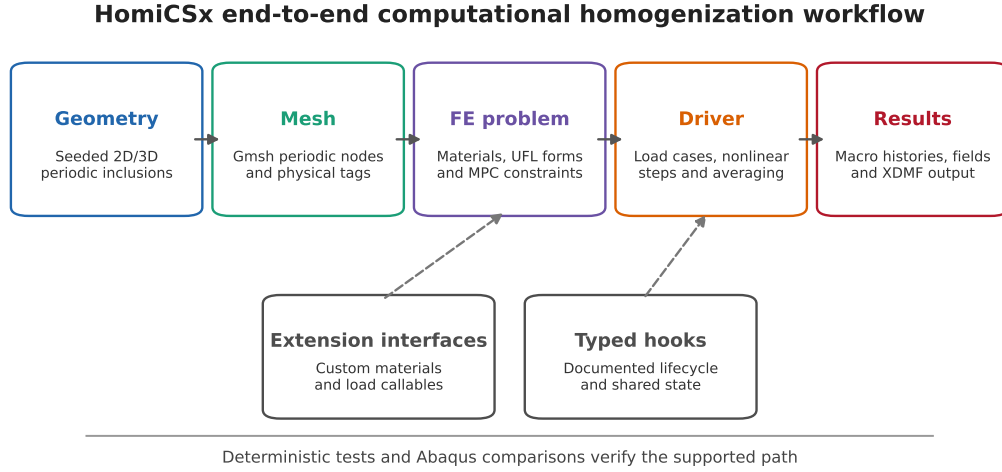


Figure 1: HomiCSx architecture and data flow. The supported workflow proceeds from geometry through periodic meshing, finite-element construction, homogenization, and result extraction. Custom materials, load callables, and typed hooks expose documented extension points.

3. Illustrative examples

The repository includes executable demonstrations for geometry generation, 2D and 3D linear homogenization, nonlinear hyperelasticity, generalized-Maxwell homogenization, mesh convergence, and hook-based field processing. In addition to narrative notebooks, compact deterministic scripts for linear and heterogeneous viscoelastic homogenization are executed by the test suite. The linear script creates a fixed geometry, generates a periodic-conforming mesh, assigns phase materials, and passes these objects to the linear driver. The viscoelastic script exercises nonlinear equilibrium, state evolution, and a typed macro-stress hook. The same separation of concerns allows a geometry or material implementation to be changed without replacing the full pipeline.

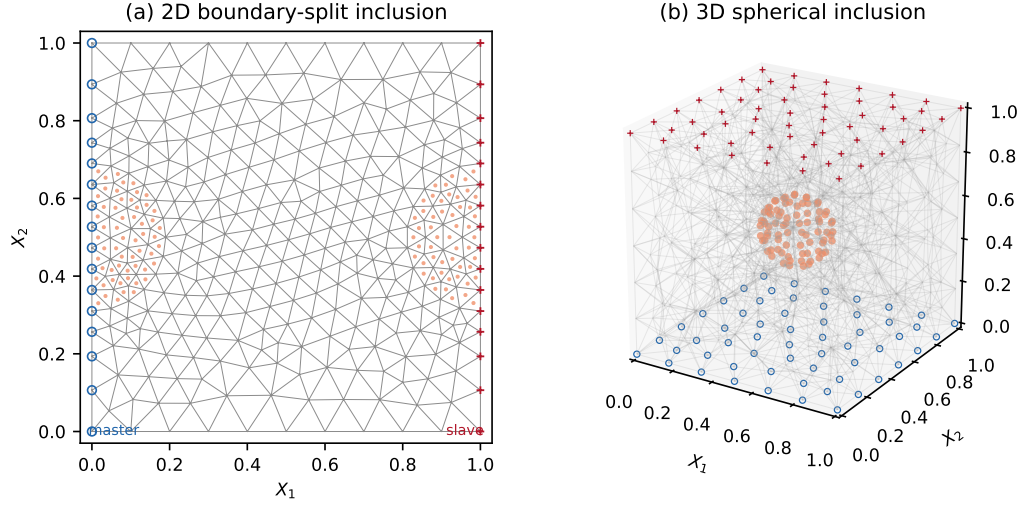


Figure 2: Representative periodic-conforming HomiCSx meshes. The 2D cell contains one physical circle split across the left/right boundary; the 3D cell contains a centered sphere. Blue circles mark master-face nodes and red crosses mark matching slave-face patterns.

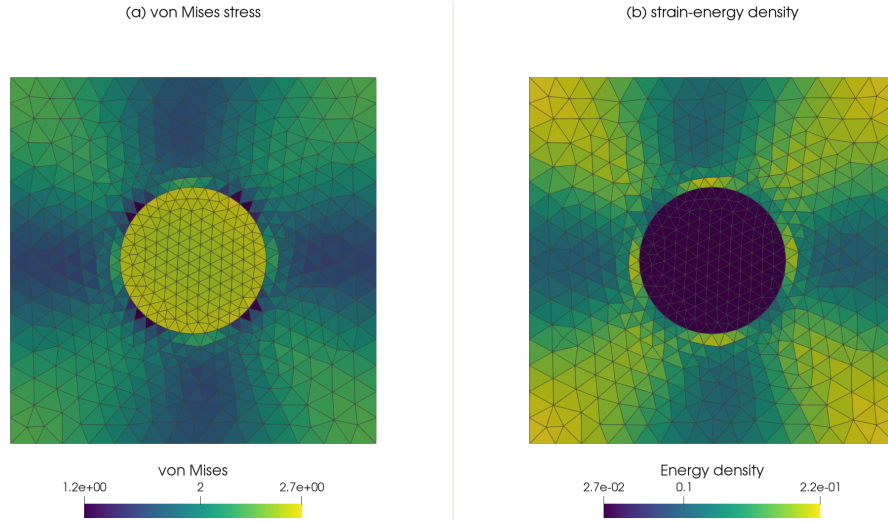


Figure 3: ParaView rendering of the final state at engineering shear $\gamma_{12} = 0.25$ for a two-phase Neo-Hookean cell. (a) von Mises stress; (b) strain-energy density. The field data are exported by HomiCSx and mesh edges are retained to distinguish the numerical field from a schematic.

4. Verification and validation

The automated test suite covers deterministic geometry behavior, tagging, periodic mesh construction, public API behavior, hook semantics, constitutive parameter validation, and solver integration. Homogeneous 2D plane-strain and six-load-case 3D linear problems are compared with analytical isotropic stiffness. A deterministic 2D refinement gate reduces relative stiffness error from 2.543% on the coarse mesh to 0.211% on the refined mesh. Neo-Hookean first Piola stress is independently checked against numerical energy derivatives in 2D and 3D, and an end-to-end nonlinear patch recovers analytical macroscopic energy, stress, and mean Jacobian.

Because periodic-boundary enforcement is a consequential implementation choice in multiscale homogenization [6], the external validation suite includes a boundary-split geometry. It uses independently generated Abaqus models and only conventional macroscopic quantities. Three linear plane-strain cases—a homogeneous non-unit cell, a centered 20% circular inclusion, and a periodic boundary-split inclusion—have maximum stiffness, probe-stress, and probe-energy differences below 0.55%. Homogeneous finite simple shear agrees to within 0.00016% for macro energy and shear stress.

Generalized-Maxwell validation compares complete macro-shear-stress relaxation histories. The homogeneous 2D and 3D curves agree with Abaqus to within 0.00063% and 0.00068% peak-normalized maximum error, respectively. For a viscoelastic matrix containing a hyperelastic inclusion, the centered and periodic-split 2D cases give 1.22% and 1.35% maximum curve errors. Automated fast/slow-rate regressions in 2D and 3D additionally require different converged heterogeneous fluctuation fields. The external evidence does not establish pointwise field identity, all loading paths, or external validation of a heterogeneous 3D viscoelastic cell.

5. Impact and reuse

HomiCSx enables a researcher to move from a seeded particulate description to homogenized linear, hyperelastic, or viscoelastic response in one inspectable workflow. Its hook mechanism is particularly relevant when a study needs quantities or interventions that are not general enough to belong in a fixed post-processing interface. The software is already used by its author in a study of matrix localization in random periodic composites; that application uses custom material and field-processing logic while retaining the common geometry, mesh, equilibrium, and homogenization pipeline.

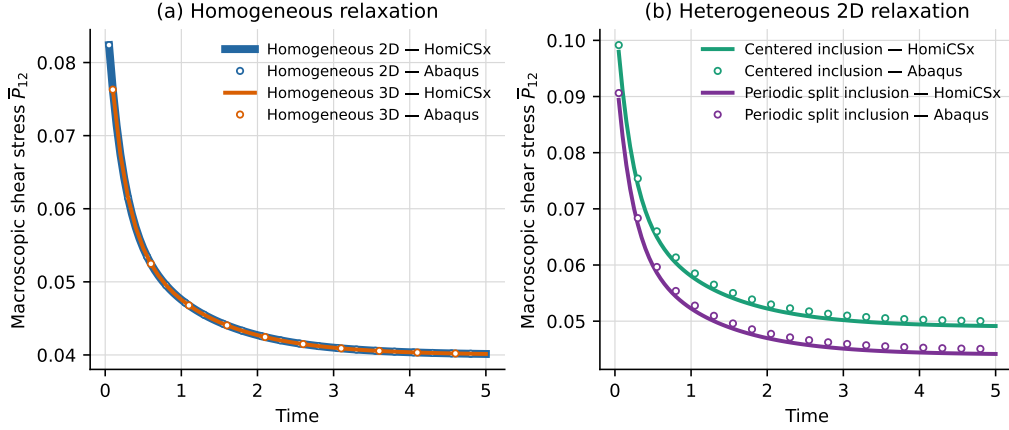


Figure 4: HomiCSx and Abaqus generalized-Maxwell shear-relaxation histories. Solid lines denote HomiCSx and open markers denote Abaqus. (a) Homogeneous 2D and 3D cells; (b) centered and periodic-split hyperelastic inclusions in a viscoelastic matrix.

Potential reuse includes parametric studies of particulate composites, development and comparison of constitutive laws, generation of training data for reduced-order or surrogate models, and teaching reproducible RVE analysis. No claim of external adoption is made at this stage. The MIT license, public interfaces, tests, and archived releases are intended to allow independent use and continuation.

6. Limitations and maintenance

The tested environment is Linux, including Linux under WSL2, with Python 3.10, DOLFINx 0.9.0, and dolfinx_mpc 0.9.0. Two-dimensional FEM uses plane strain; plane stress is not implemented. The supported meshing path uses triangles and tetrahedra. Imported meshes, overlapping-void and open-cell workflows, advanced visualization, and stochastic convenience modules are outside the publication-supported core. The latter two modules are explicitly experimental.

HomiCSx is maintained as archival research software. No feature schedule or guaranteed support response is promised. Reproducibility, documentation, correctness fixes as resources permit, versioned releases, and preservation are prioritized. The permissive license allows community forks and continued development if active maintenance changes.

7. Availability and reproducibility

Source code: <https://github.com/AmirTahouni/HomiCSx>

Documentation: <https://homicsx.readthedocs.io/en/latest/>

License: MIT

Version described: To be assigned at release

Archive DOI: To be assigned at archival release

The repository contains pinned Conda environment specifications, installation instructions, automated tests, benchmark manifests, independent Abaqus runner scripts, compact reference results, and executable acceptance gates. Abaqus is not required to recompute comparisons from the committed compact results.

Software metadata

Current code version	To be assigned at release
Permanent link to code/repository	To be assigned at archival release
Code repository	https://github.com/AmirTahouni/HomiCSx
Legal software license	MIT
Code versioning system	Git
Languages and tools	Python, UFL, Gmsh API
Installation requirements	Conda; Python 3.10; DOLFINx 0.9.0; dolfinx_mpc 0.9.0
Operating environment	Linux; WSL2 tested
User documentation	https://homicsx.readthedocs.io/en/latest/
Support email	tahouniamirreza@gmail.com

Table 1: Software metadata for the publication-supported HomiCSx release.

CRedit authorship contribution statement

Amir Reza Tahouni: Conceptualization, Methodology, Software, Validation, Investigation, Data curation, Visualization, Writing—original draft, Writing—review and editing.

Funding

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

Declaration of competing interest

The author declares no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

The software, validation manifests, scripts, and compact numerical results are available in the public repository and will be preserved in the archived release identified above. Large proprietary Abaqus working databases are not required to evaluate the committed comparison results and are not distributed.

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

During the preparation of this work, the author used an OpenAI coding assistant to support code review, test and validation development, documentation editing, and manuscript drafting. After using this service, the author reviewed and edited the content as needed and takes full responsibility for the content of the publication.

References

- [1] I. A. Baratta, J. P. Dean, J. S. Dokken, et al., DOLFINx: The next generation FEniCS problem solving environment, preprint (2025). <https://doi.org/10.5281/zenodo.18101307>.
- [2] M. S. Alnæs, A. Logg, K. B. Ølgaard, M. E. Rognes, G. N. Wells, Unified Form Language: A domain-specific language for weak formulations of partial differential equations, *ACM Trans. Math. Softw.* 40 (2014). <https://doi.org/10.1145/2566630>.

- [3] C. Geuzaine, J.-F. Remacle, Gmsh: A 3-D finite element mesh generator with built-in pre- and post-processing facilities, *Int. J. Numer. Methods Eng.* 79 (11) (2009) 1309–1331. <https://doi.org/10.1002/nme.2579>.
- [4] M. G. D. Geers, V. G. Kouznetsova, W. A. M. Brekelmans, Multi-scale first-order and second-order computational homogenization of microstructures towards continua, *Int. J. Multiscale Comput. Eng.* 1 (4) (2003) 371–386. <https://doi.org/10.1615/IntJMultCompEng.v1.i4.40>.
- [5] S. Saeb, P. Steinmann, A. Javili, Aspects of computational homogenization at finite deformations: A unifying review from Reuss’ to Voigt’s bound, *Appl. Mech. Rev.* 68 (5) (2016) 050801. <https://doi.org/10.1115/1.4034024>.
- [6] P. Henyš, L. Čapek, J. Březina, Comparison of current methods for implementing periodic boundary conditions in multi-scale homogenisation, *Eur. J. Mech. A Solids* 78 (2019) 103825. <https://doi.org/10.1016/j.euromechsol.2019.103825>.
- [7] A. R. Tahouni, HomiCSx, archival release, Zenodo (2026). DOI to be assigned at archival release.